

BUGTRIAGE AI

Clasificación asistida de reportes de errores mediante un modelo de IA
preentrenado

Trabajo práctico integrador

Seminario de Actualización 2026 · IFTS N.º 18

Autora: Gabriela Coronel

Aplicación publicada: [Hugging Face Spaces](#) — BugTriage AI

Objetivo: construir, evaluar y desplegar una aplicación funcional que use inferencia con modelos preentrenados, sin entrenar un modelo propio.

Documento elaborado para explicar el problema, la elección del modelo, la arquitectura, el despliegue, las dificultades y los límites de la solución.

A. El problema

Los equipos de desarrollo reciben reportes de errores escritos por usuarios, testers o integrantes del equipo. Los reportes pueden ser breves, informales, ambiguos o mezclar más de un problema. Antes de priorizarlos, alguien debe leerlos, interpretarlos y derivarlos al área adecuada. Ese trabajo manual puede demorar el triage y producir criterios inconsistentes.

Problema concreto

¿Cómo ayudar a clasificar rápidamente un reporte de bug en una categoría inicial para facilitar su derivación, sin reemplazar el criterio de una persona?

Solución propuesta

BugTriage AI es una aplicación web que recibe un texto y propone una categoría entre cinco etiquetas: Funcional, UI/UX, Autenticación, Datos y Accesibilidad. La interfaz también presenta una puntuación del modelo, una estimación orientativa de nivel y categorías alternativas para ayudar a revisar la decisión.

Alcance y límites

El prototipo clasifica texto; no reproduce el error, no inspecciona código fuente, no crea tickets automáticamente y no determina de forma fiable la severidad real del incidente. La categoría sugerida es una primera orientación y requiere validación humana, sobre todo cuando el reporte es ambiguo o afecta varias áreas.

Usuarios destinatarios

Personas de QA, soporte técnico y desarrollo que necesitan ordenar reportes antes de asignarlos. En una futura integración, podría funcionar como etapa preliminar de un sistema de tickets, siempre con mecanismos de corrección y trazabilidad.

B. El modelo

La aplicación usa clasificación zero-shot: en lugar de entrenar un clasificador propio con un conjunto etiquetado, se le entrega un texto y un conjunto de etiquetas candidatas. El modelo estima qué etiquetas se relacionan mejor con el contenido. En el proyecto se compararon dos modelos de inferencia preentrenados con los mismos 12 casos.

Modelos evaluados

Modelo	Accuracy reportada	Tiempo medio/caso
facebook/bart-large-mnli	50,00 % (6/12)	0,90 s
MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7	16,67 % (2/12)	4,15 s

Decisión

Se seleccionó facebook/bart-large-mnli porque obtuvo el mejor resultado observado y menor tiempo medio en este benchmark concreto. Aunque el segundo modelo es multilingüe, en estas pruebas rindió peor. Esta decisión no demuestra que BART sea superior en general: depende de las etiquetas, la redacción y la composición de los casos evaluados.

Cómo interpretar los resultados

La muestra tiene solo 12 ejemplos; por eso, una diferencia de uno o dos aciertos cambia mucho el porcentaje. Los valores son evidencia exploratoria para elegir una opción de prototipo, no una medición definitiva de calidad en producción. La puntuación que muestra la interfaz es un score relativo del modelo, no una probabilidad calibrada de que la categoría sea correcta.

Próxima mejora

Ampliar el conjunto de prueba, definir criterios de etiquetado, incluir más variantes de español rioplatense, separar ejemplos de desarrollo y evaluación, y medir por categoría precisión, recall y matriz de confusión. Registrar errores ambiguos y validar las etiquetas con más de una persona.

C. Dónde corre cada cosa

La solución separa la interfaz web del proceso de inferencia, aunque en el despliegue actual la aplicación de Gradio y la carga del modelo están coordinadas desde el mismo proyecto.

Componentes

Componente	Responsabilidad	Dónde corre
Interfaz Gradio	Recibe el reporte, permite enviarlo y muestra categoría, score y alternativas.	Hugging Face Space
Pipeline Transformers	Prepara la inferencia zero-shot con las etiquetas candidatas.	Entorno Python del Space
Modelo BART MNLI	Calcula scores de compatibilidad entre el texto y las etiquetas.	Infraestructura de inferencia asignada al Space
Notebook de evaluación	Compara modelos con casos de prueba, calcula métricas y registra resultados.	Entorno local de desarrollo

Flujo de una consulta

1. La persona escribe un reporte. 2. La interfaz entrega el texto al pipeline. 3. El modelo compara el texto con las categorías candidatas. 4. La aplicación ordena los scores y construye la respuesta. 5. La persona revisa la sugerencia y decide cómo proceder.

Inferencia local frente a inferencia remota

Durante la evaluación se probó el uso de un cliente remoto de inferencia, además de la vía con pipeline. La ruta que quedó funcionando en la aplicación usa el pipeline de Transformers dentro del entorno del Space. La prueba remota sirvió para identificar diferencias entre las interfaces de cliente, las credenciales y la disponibilidad de créditos del servicio.

Seguridad básica

Los tokens de acceso no deben escribirse en el código ni subirse al repositorio. Cuando una API necesita autenticación, el secreto debe configurarse como variable de entorno o secreto del servicio. Los mensajes de error deben evitar exponer credenciales o datos sensibles de los reportes.

D. El deploy

BugTriage AI está publicado como una aplicación Gradio en Hugging Face Spaces. El repositorio del Space contiene los archivos necesarios para construir y ejecutar la interfaz. La configuración de ejecución usa ZeroGPU a través de la configuración de Spaces del proyecto.

Enlace público

<https://huggingface.co/spaces/lagabyok/bugtriage-ai>

Archivos centrales

app.py: interfaz y lógica de clasificación. **requirements.txt**: dependencias Python. **README.md**: descripción e instrucciones del Space. **notebook/**: evaluación comparativa y archivos CSV con los resultados.

Proceso de publicación

El código se versiona con Git y se sincroniza con el repositorio del Space de Hugging Face. Los commits documentan la implementación, la configuración del Space, la actualización del README y la evaluación de modelos. El repositorio de GitHub del seminario es el destino académico para la entrega completa; conviene mantener separados los cambios de la aplicación desplegada y los materiales de evaluación/informe.

Prueba de funcionamiento

La app acepta texto en español y devuelve una categoría sugerida junto con información auxiliar. Una prueba con “El botón de guardar no funciona” mostró que la etiqueta de mayor score puede no coincidir con la interpretación humana más intuitiva. Este caso es útil para explicar por qué no debe tratarse la salida como una decisión definitiva.

Reproducibilidad

Para reproducir la aplicación se necesitan Python, las dependencias listadas en requirements.txt y acceso a los recursos de inferencia requeridos por la configuración del Space. El notebook de evaluación registra los casos usados y los resultados obtenidos; las métricas pueden variar con cambios de versión, hardware, caché o condiciones de ejecución.

E. Lo que se rompió

Durante el desarrollo surgieron problemas de configuración y disponibilidad que ayudaron a distinguir entre fallas de código, autenticación y límites del servicio.

1. Variable de entorno ausente

La primera prueba con el cliente remoto indicó que HF_TOKEN no estaba definido en el entorno. La lección fue verificar la configuración antes de inicializar el cliente y nunca imprimir ni guardar el token en archivos versionados.

2. Error 401: Unauthorized

Después se obtuvo una respuesta 401 porque el cliente se inicializaba sin un token válido. Esto no indicaba necesariamente un problema en el modelo o en el texto de entrada; era un problema de credenciales/permisos.

3. Error 402: Payment Required

Luego apareció un error 402 por falta de créditos de inferencia disponibles. La conclusión es que una API remota puede depender de cuotas, permisos y condiciones de cuenta. Para un prototipo académico, hay que documentar esa dependencia y disponer de una ruta de ejecución alternativa.

4. Tarea incorrecta del cliente

La primera llamada probó text_classification, pero la tarea del modelo y el uso previsto requerían zero-shot classification. Se corrigió la llamada para usar zero_shot_classification con candidate_labels y multi_label=False. La interfaz del cliente debe coincidir con la tarea declarada por el modelo.

Qué quedó funcionando y qué falta

La aplicación se apoya en el pipeline que ya funciona en el Space. La ruta de cliente remoto quedó como experimento de evaluación y no debe considerarse indispensable para que el usuario final utilice la app. Queda como mejora incorporar pruebas automatizadas, manejar con claridad los errores de inferencia y ampliar la evaluación.

F. Vocabulario

Inferencia. Uso de un modelo ya entrenado para obtener una salida a partir de una entrada nueva.

Modelo preentrenado. Modelo cuyos parámetros se aprendieron previamente; el proyecto lo utiliza sin entrenarlo desde cero.

Zero-shot classification. Clasificación en la que se comparan textos con etiquetas candidatas sin ajustar el modelo con un dataset propio específico.

NLI (Natural Language Inference). Tarea de inferencia entre enunciados; algunos modelos NLI pueden reutilizarse para comparar una premisa con hipótesis asociadas a etiquetas.

Pipeline. Abstracción de Transformers que reúne la preparación de entrada, la inferencia y el postprocesamiento.

Hugging Face Spaces. Servicio para alojar y publicar demostraciones de aplicaciones de machine learning.

Gradio. Biblioteca Python para construir interfaces web interactivas para modelos y funciones.

API / cliente remoto. Interfaz que permite solicitar inferencia a un servicio externo a través de una llamada programática.

Token de acceso. Credencial que permite autenticar una petición; debe guardarse como secreto y no publicarse.

Accuracy. Proporción de predicciones correctas entre todos los casos evaluados.

Benchmark. Conjunto de pruebas y procedimiento para comparar alternativas bajo condiciones similares.

Score. Valor de puntuación producido por el modelo. No equivale necesariamente a una probabilidad calibrada.

Triage. Proceso inicial de revisar, categorizar y derivar incidencias.

Deploy. Publicación de una aplicación en un entorno donde otras personas pueden acceder a ella.

Declaración de uso de IA generativa

Se utilizaron herramientas de IA generativa como apoyo para organizar y revisar la documentación del proyecto y para repasar conceptos técnicos. La implementación, las pruebas, la selección de los modelos y la revisión de los resultados se contrastaron con el código y las salidas del proyecto.

La herramienta no reemplaza la responsabilidad sobre la veracidad, comprensión y defensa del trabajo.

Conclusión

BugTriage AI demuestra un flujo de inferencia con un modelo preentrenado, una interfaz web y un despliegue público. La comparación inicial permitió elegir un modelo para el prototipo, pero también reveló límites de calidad, ambigüedad y dependencia de servicios. Durante las pruebas observé que la clasificación puede presentar errores y que algunos reportes son ambiguos.

También comprobé que utilizar un servicio externo de inferencia puede tener restricciones, como la falta de créditos.

Por eso, mi objetivo no es afirmar que la aplicación clasifica todos los errores correctamente, sino demostrar un

prototipo funcional que pueda seguir mejorándose mediante más pruebas, métricas y revisión humana.